Supervised tasks

UNIVERSITÀ DI PISA

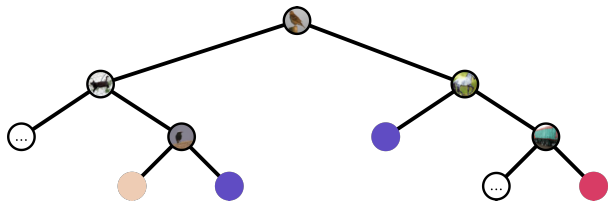
Three approaches to classification

Università di Pisa

By any means, not the only ones!

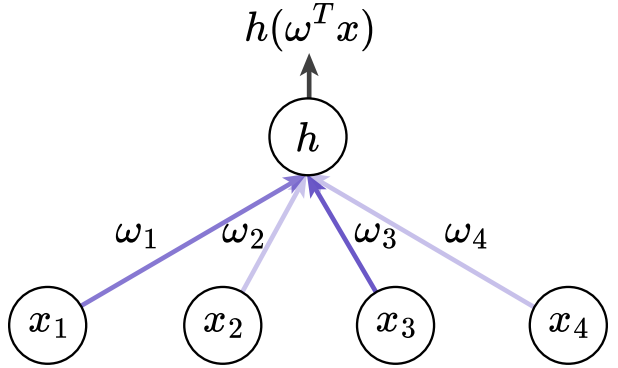
Trees

Can I learn a space partition that separates the data according to its label?

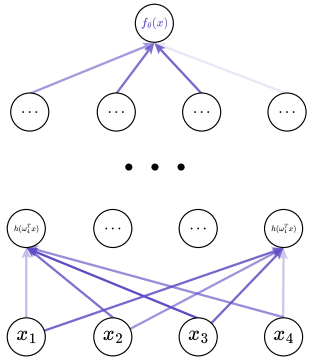


Linear

Can I predict the label through linear modeling?



Neural



Trees

UNIVERSITÀ DI PISA

Yet again

Splitting trees

UNIVERSITÀ DI PISA

Inducing Decision Trees depends roughly on two factors:

- Split function f_i: how do I route instances in my subtrees?
- Split loss: how do I evaluate the quality of a split?

We are considering binary trees.

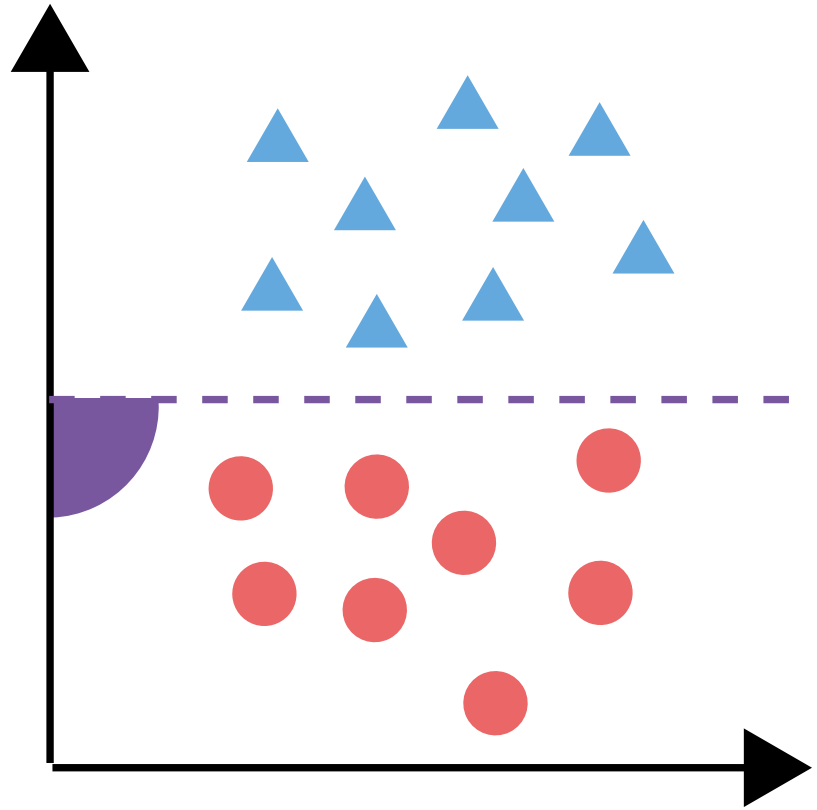
Split function: Univariate

The de facto standard:

$$f_i(x) \equiv x_i \leq \theta_i.$$

- Highly interpretable
- Easy to control complexity

- Limited capacity



A univariate split: the split is orthogonal to the axes.

Split function: Multivariate

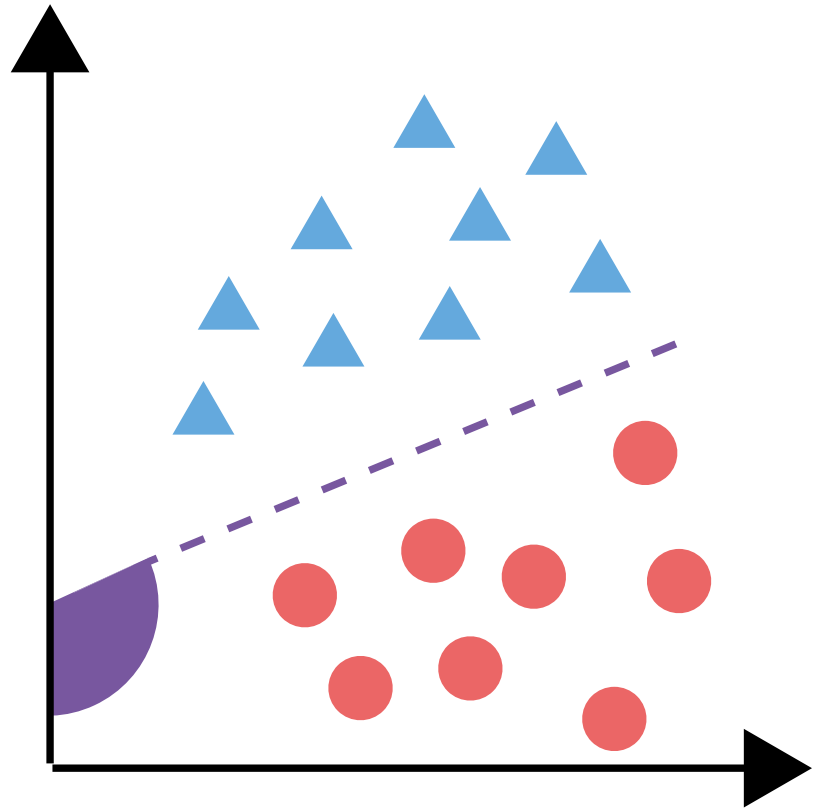
A higher-capacity model, with a more expressive split function:

$$f_i(x) \equiv x_i\theta_i + \theta_i^0$$

Since they generate "oblique" (non axis-parallel) splitting hyperplanes, they are also called *oblique* trees.

- Shallower tree
- Higher capacity

- Dubious interpretabilty



A multivariate split: the split is not orthogonal to the axes. Also called *oblique* split.

Decision Tree SVM: An extension of linear SVM for non-linear classification. Nie et al.

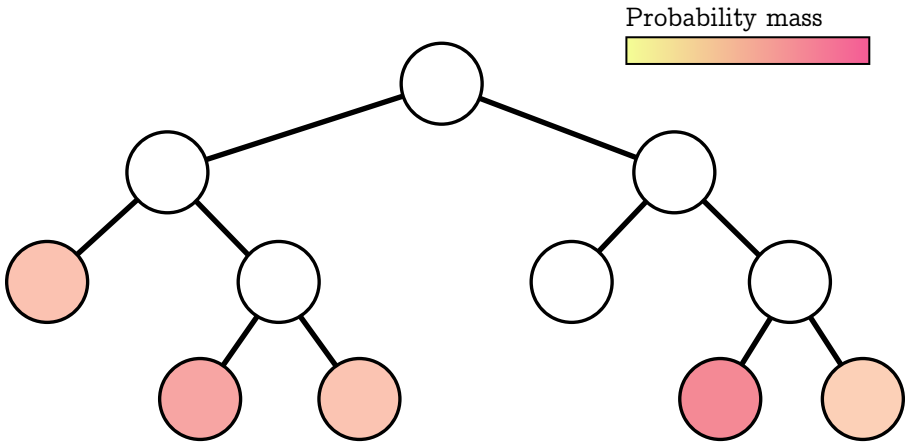
Split function: Probabilistic

Università di Pisa

Probabilistic routing defined by a learned probability distribution p_i:

$$f_i(x) \equiv p_i(x; \theta_i).$$

Instances are routed to all leaves in the tree, accumulating probability density at each node. The prediction can then be given by the leaf with higher probability mass, or average class probabilities across leaves.



A probabilistic tree: an instance is routed to each leaf, accumulating probability mass along the path.

Distilling a Neural Network Into a Soft Decision Tree, Frosst and Hinton.

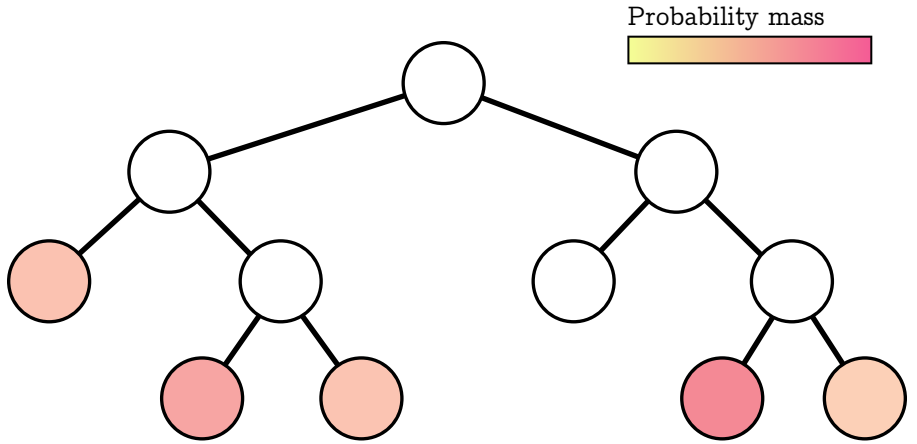
Split function: Probabilistic

Università di Pisa

Instances are routed to all leaves in the tree, accumulating probability density at each node. The prediction can then be given by the leaf with higher probability mass, or average class probabilities across leaves.

- High flexibility
- Better on non-relational data, e.g., images

- Lower interpretabilty



A probabilistic tree: an instance is routed to each leaf, accumulating probability mass along the path.

Distilling a Neural Network Into a Soft Decision Tree, Frosst and Hinton.

Linear models

UNIVERSITÀ DI PISA

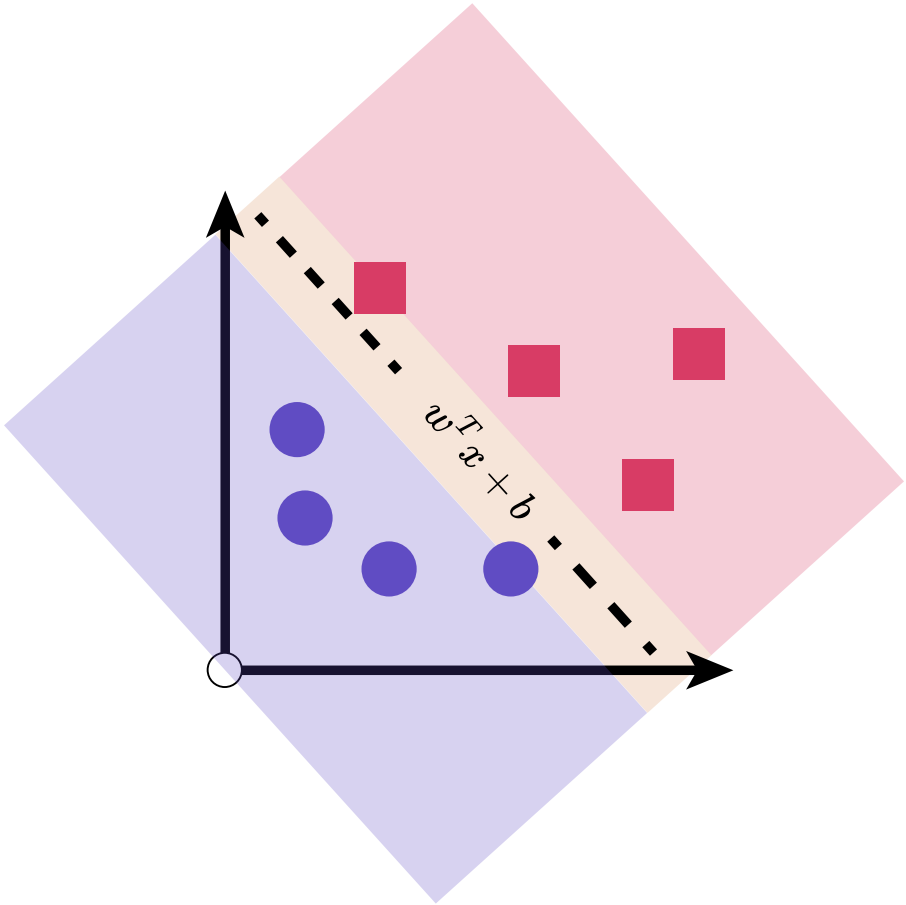
Or are they?

Distilling a Neural Network Into a Soft Decision Tree, Frosst and Hinton.

Linear modeling

Linear models are simple, generally interpretable, family of models. Typically used for tasks on

- Representation, e.g., PCA, PLA: can I find an alternative, *linear* representation of my data?
- Classification, e.g., Linear regression: can I predict a variable as a *linear* model of my data?



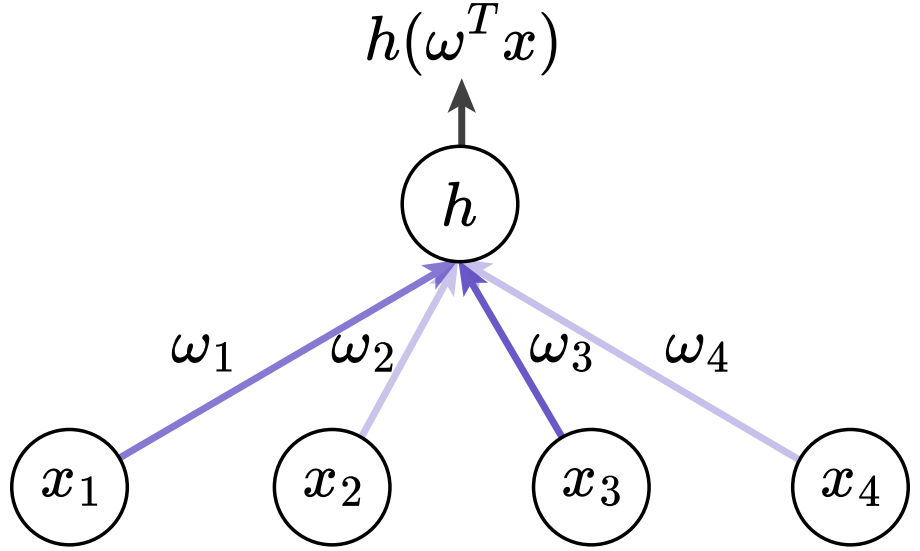
Dataset of two classes, and a linear model separating them.

Linear models for classification

A linear model parameterized by ω has form

$$f \equiv \omega^T x,$$

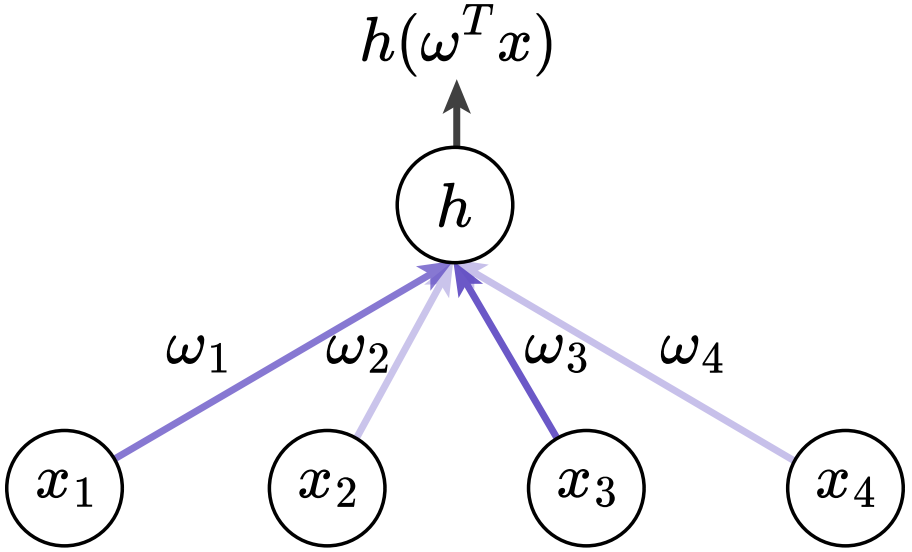
and can be leveraged for classification with a suitable activation function, e.g., $f(x) \equiv \sigma(\omega^T x)$.



An architectural illustration of a linear model. Note: such models include a bias term, not included in the figure.

Linear models for classification

Università di Pisa



An architectural illustration of a linear model. Note: such models include a bias term, not included in the figure.

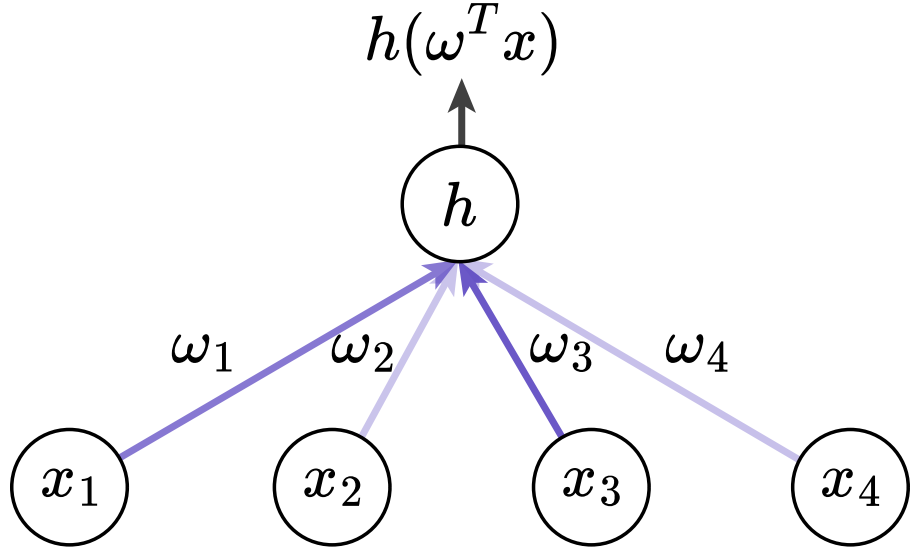
- Interpretable, somewhat actionable
- Ease of optimization
- Strong baseline

- Limited capacity

From linear... to additive

The interpretability of linear models comes from their *additive* nature: each component has a measurable and independent contribution $\omega_i x_i$.

Their limitation also comes from this, as there is no real learned representation of the data.



An architectural illustration of a linear model. Note: such models include a bias term, not included in the figure.

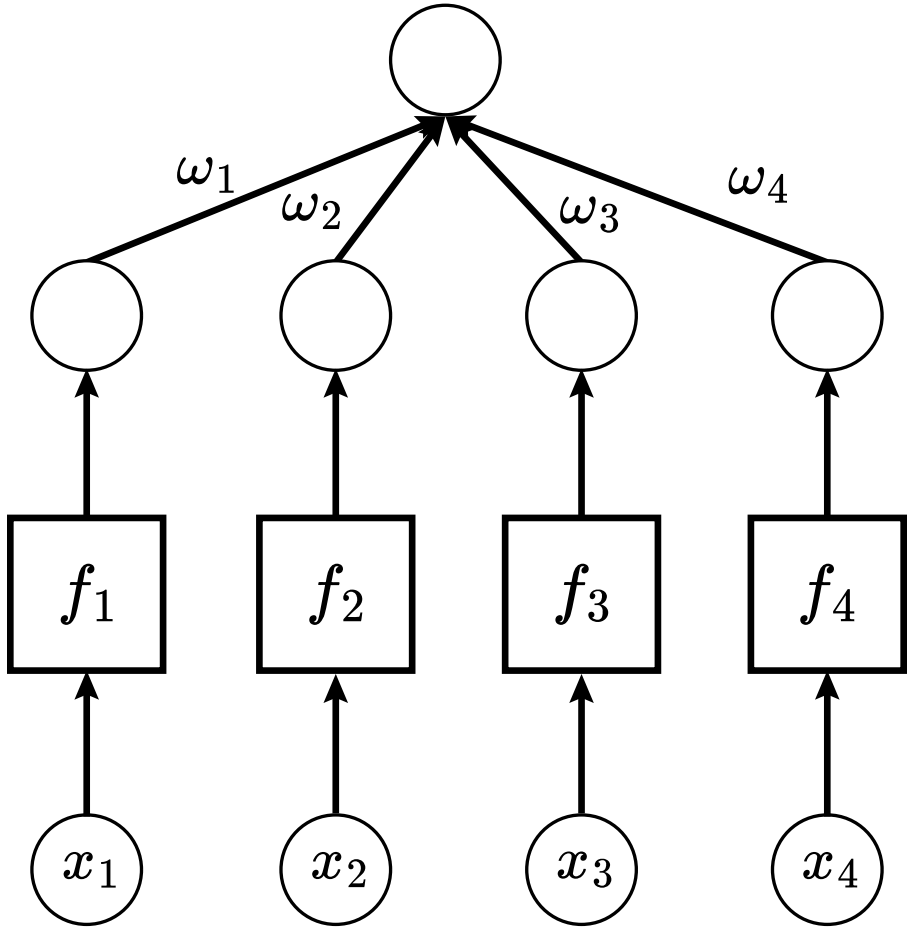
Note: *independency* in the model, not necesarilly in the data!

Generalized Additive Models

Generalized Additive Models (GAMs) generalize linear models by introducing *per-feature* learned representations:

$$f(x) \equiv \sum^{m} \omega_i f_i(x_i) + \omega_0.$$

Each feature enjoys a *learned* representation given by a parametric *shape* function f_i of arbitrary complexity.



An architectural illustration of a GAM. Each block f_i indicates a differentiable shape function.

Generalized additive models, D. F. McCaffrey.

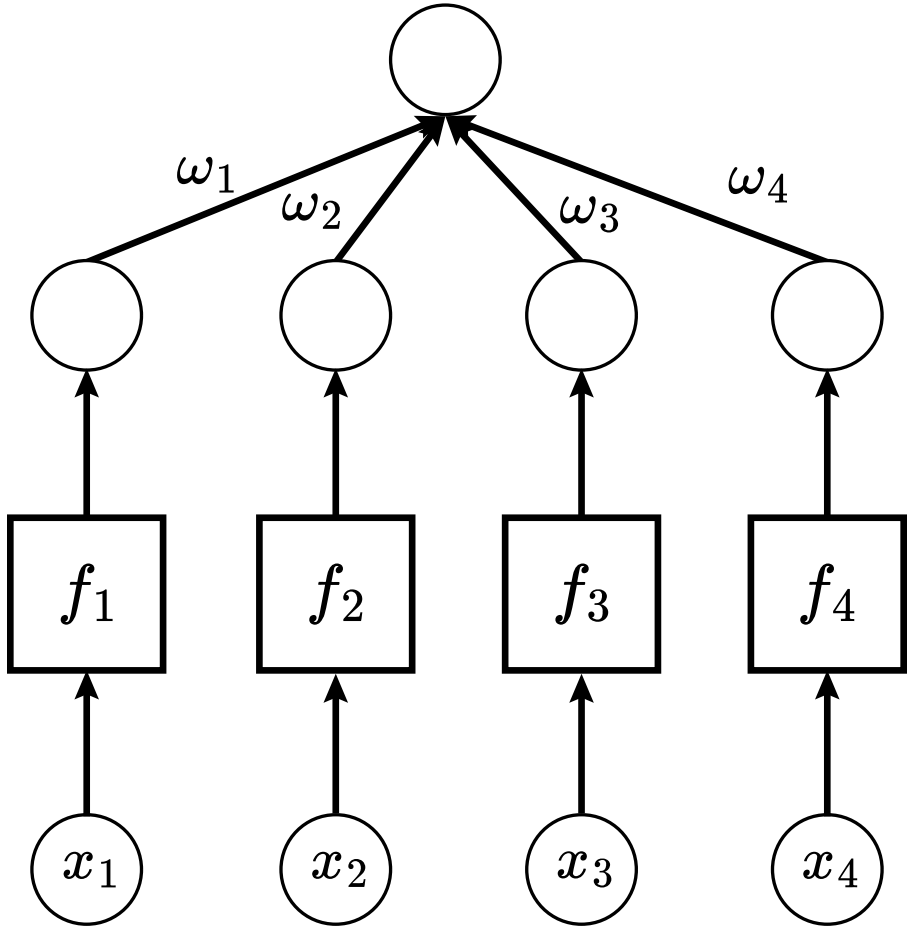
Generalized *Additive* Models

Università di Pisa

$$f(x) \equiv \sum^{m} \omega_i f_i(x_i) + \omega_0.$$

Typical shape functions:

- identity (linear models)
- neural networks



An architectural illustration of a GAM.

Ensemble methods

UNIVERSITÀ DI PISA

Tackling variance, once more

Mitigating variance... or not

As highlighted by the bias-variance decomposition, learning algorithms can incur in a variance in terms of generalization. We have seen two strategies to mitigate this:

- Cross validation: reduce variance by increasing samples
- Regularization: reduce variance by reducing capacity

There exists a third way, somewhat related, approach: leverage variance.

Bagging

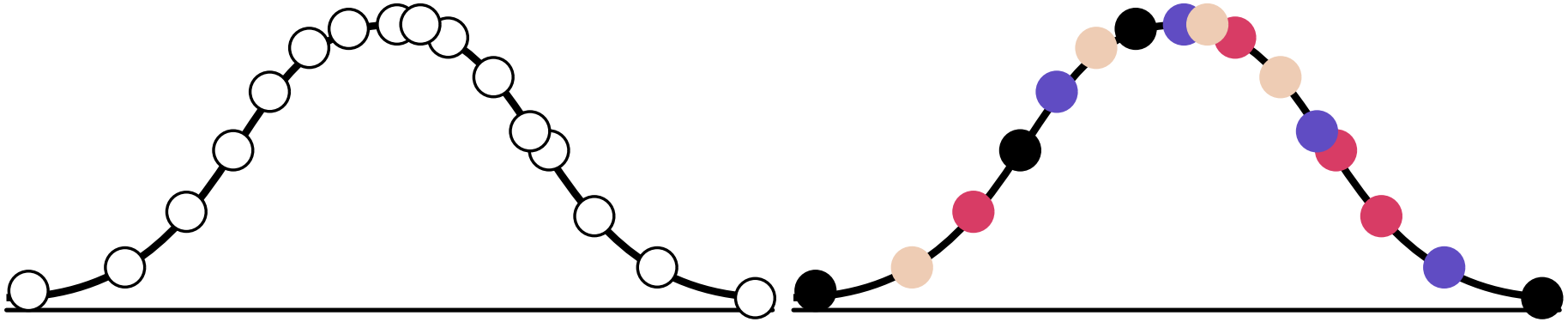
Bagging leverages learning algorithms with **moderate-to-high variance**: rather than learn a single model f_θ, it learns a set of models $f_1, \ldots, f_T$, and combines them into one. A bagging model is of the form

$$f_\theta(x) = \sum_{i=1}^{T} \eta f_i(x), \qquad \eta \in \mathbb{R}.$$

Following the bias-variance, bagging algorithms aim to learn a model on separate samples of the original distribution.

Assumption: models retain a low-enough bias to be accurate on their respective bags.

Bagging



On the left, data, sampled from the data distribution. On the right, a visualization of bagging: samples are split in several bags (here, color-coded). Each bag is used to learn a different model, which will then be part of an ensemble.

Bagging: computational complexity

Given by the computational complexity of each model. For a uniform learning time of $\mathcal{O}(c)$ of k bags, the cost is simply $\mathcal{O}(kc)$, hence $\mathcal{O}(c)$.

For non-uniform costs, the computational complexity is upper-bounded by $\max_{c \in C} \mathcal{O}(c)$, hence it is again $\max_{c \in C} \mathcal{O}(c)$.

Remember: in asymptotical terms, constants are not considered.

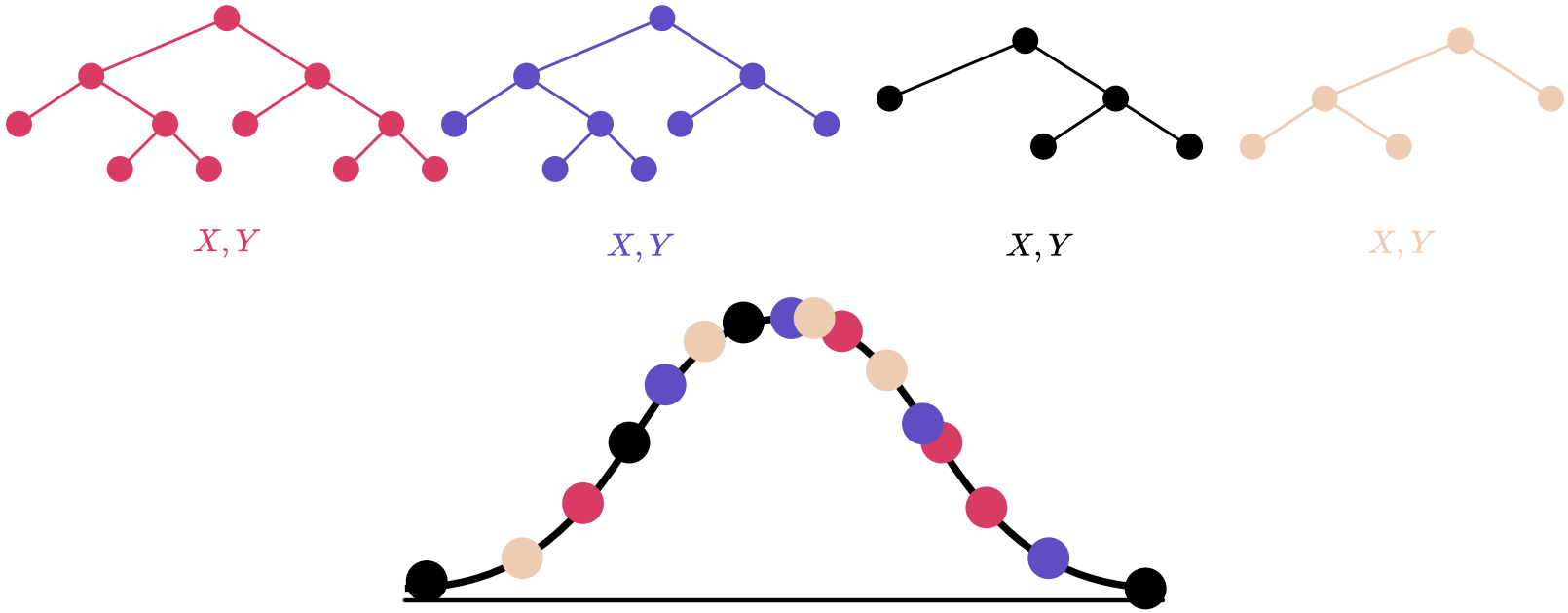
Random Forests

Decision Trees show a moderate variance, and thus are a perfect candidate. Random Forests sample a set of *bags* by sampling:

- Random instances, through stratification
- Random features within the bag

Thus creating learning sets heterogeneous in both instances and features. Bags are sampled with replacement!

Bagging



On the bottom, the bagged data, each bag indicated by a different color. On top, a set of learned trees, color-coded as the bag they have been learned from: the red tree has been learned on the red bag, and so forth. Note: Random Forest samples bags with replacement: an instance may be part of 0 up to K bags.

Random forests: computational complexity

As a bagging model, the computational complexity is given by the complexity of the single tree.

Remember: in asymptotical terms, constants are not considered.

Random Forests

UNIVERSITÀ DI PISA

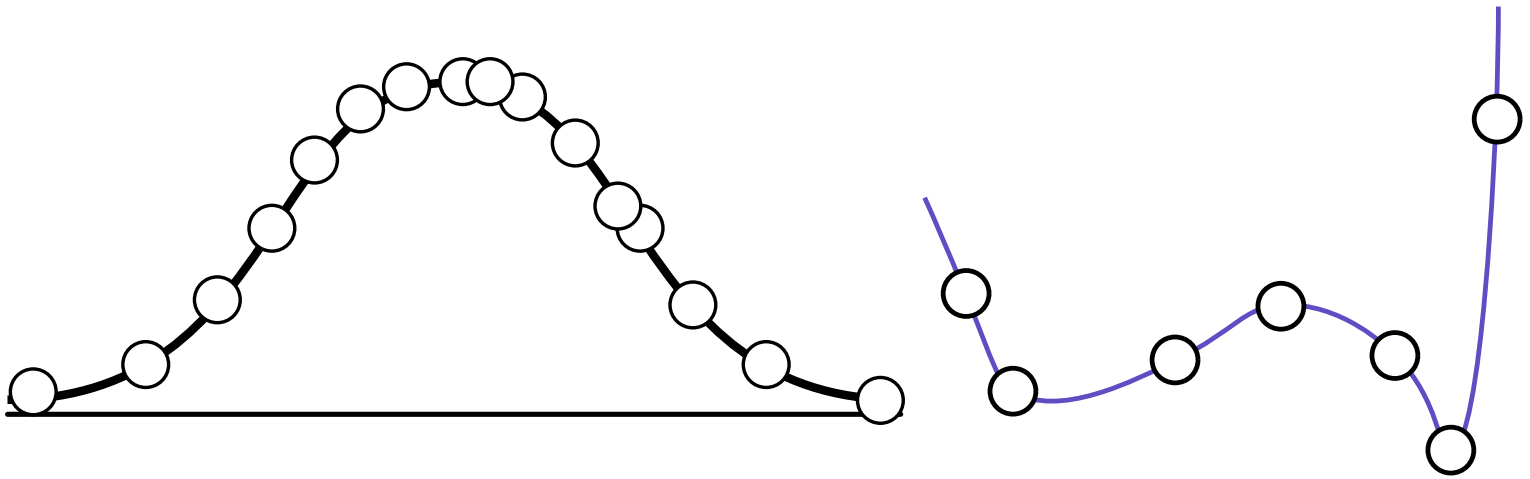
- Simple definition with high variance models
- Interpretable-ish results
- Fast learning

- Weak to high degrees of covariance
- Sampling *with replacement*: risk of high correlation
- Random bagging

Bagging... what?

Università di Pisa

How would *you* bag?



From bagging... to Boosting

Università di Pisa

With bagging models, we assume locality in the data distribution, and thus learn different models on different bags. Up to re-sampling, each instance is uniquely predicted by one model. If models have high-enough bias, then we are not gaining much from bagging data.

The *hard* bags of bagging are limited by the bias of each model: if none of them are good enough, their number does not matter, and dividing the task yields no advantage. In other words, **in a crowd of weak learners, there's no wisdom to be had**.

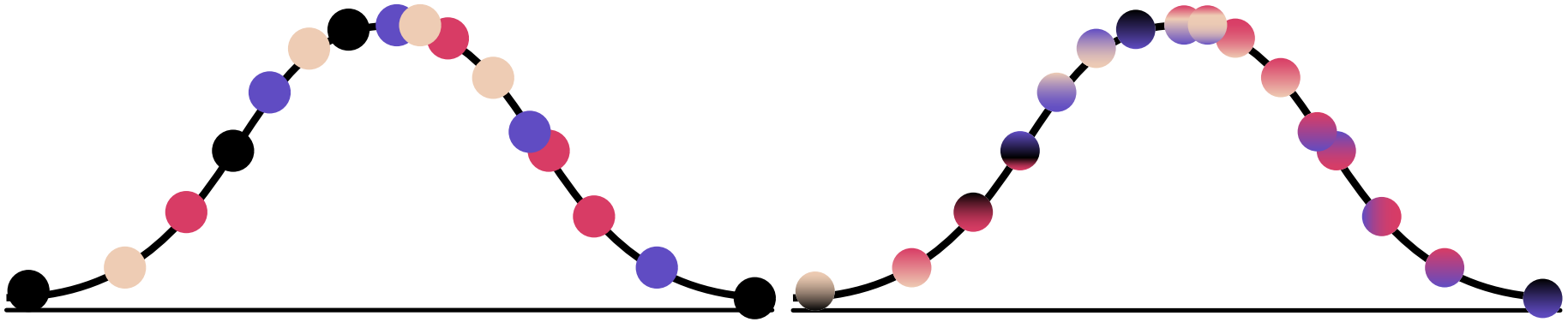
Why don't we bag the task instead?

Robert E. Schapire, The Strength of Weak Learnability.

Boosting

UNIVERSITÀ DI PISA

Boosting creates *fuzzy soft* bags, wherein instances are jointly predicted by all models, thus **decomposing the task**, rather than the data!



Bagging VS Boosting. In bagging, each bag is used to learn a model. In boosting, bags are *fuzzy*, and the task is decomposed so that each model predicts a subset of it.

Boosting: a general formulation

Like bagging, we have a model of the form

$$f_T(x) = \sum_{i=1}^{T} \eta_i f^i(x), \eta_i \in \mathbb{R},$$

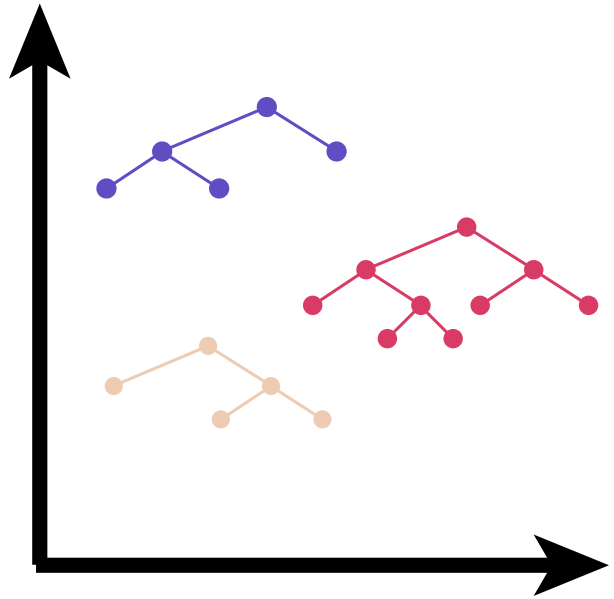
which is learned iteratively: first f_1, then f_2, etc. Sums and multiplications by scalar: looks like a job for linear algebra!

Boosting and the functional (linear) space

UNIVERSITÀ DI PISA

Since we are operating with finite datasets, we can interpret functions as vectors $\vec{f}_i = [f_i(x_1), \ldots, f_i(x_n)]$ in a vector space. Thus, we can interpret operations on functions as operations on vectors:

- $f + g = \vec{f} + \vec{g}$
- $\eta f = \eta \vec{f}$
- $f \cdot g = \vec{f} \cdot \vec{g}$



A model space of decision trees: in this space, summing allows us to generate novel decision trees, while inner products allows us to compute their alignment.

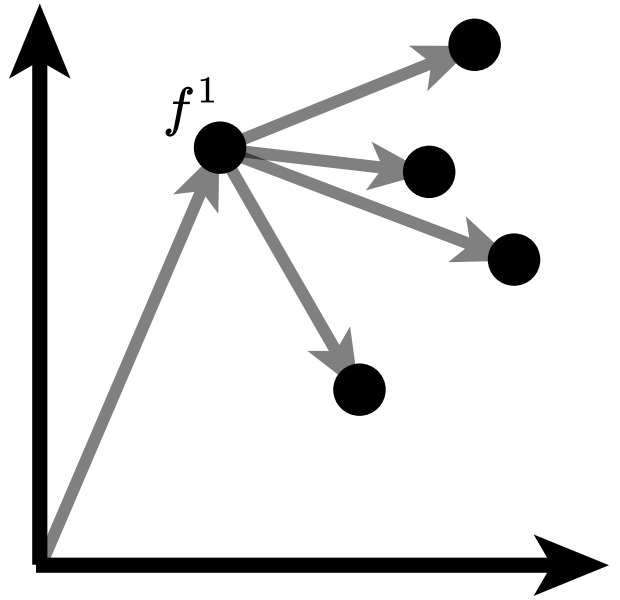
Boosting: exploring the functional space

Università di Pisa

Boosting models are learned iteratively, each additional model f_{t+1} looking to reduce a predefined loss

$$L_{t+1} = l(Y_i, f_t(x) + \eta_{t+1} f_{t+1}).$$

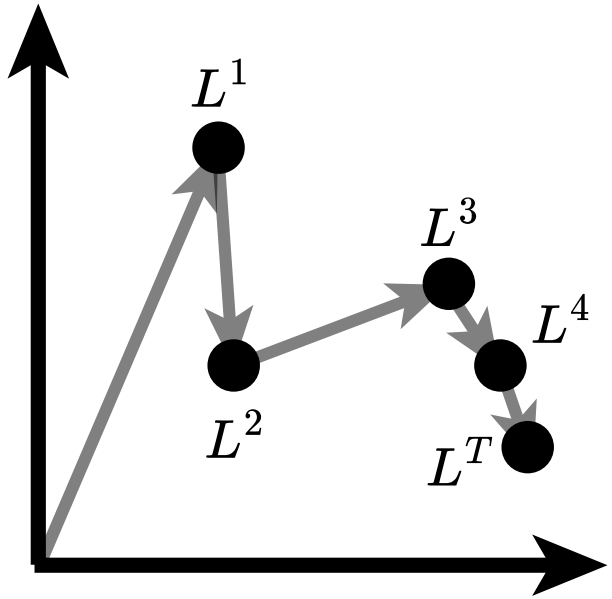
The additional function f_{t+1} is one of possibly many (possibly infinite) directions we can take in functional space. We want to pick the direction minimizing loss! What direction minimizes loss?



f_1 and some (of possibly infinite) directions f_2.

Exploring optimization: the gradient

The one **minimizing** L^{t+1}! To know such direction, we rely on calculus, i.e., we compute the gradient of L^{t+1} w.r.t. f^t : $\nabla_{f_T} L^{t+1}$. The negated gradient is the direction of minimization.



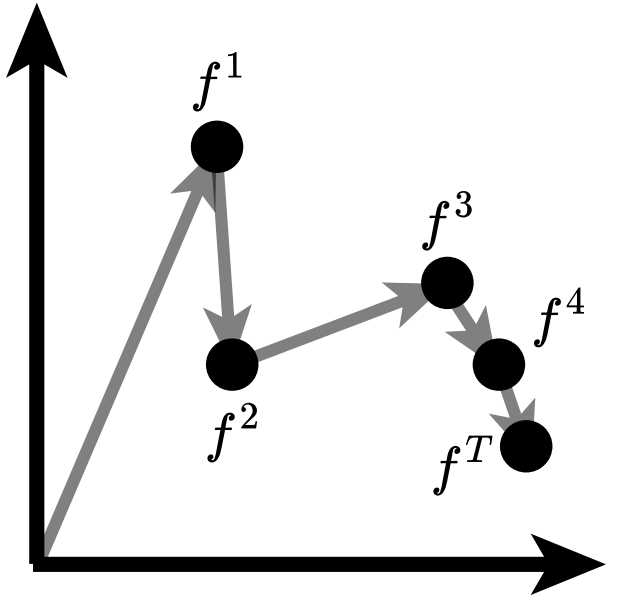
Boosting and iterative optimization: we construct the model one loss improvement at a time, exploring the loss space.

Exploring optimization: the gradient

Università di Pisa

Note: the space of models $\mathcal{F}$ to which f_{t+1} belongs may not be continuous, thus we can't necessarily directly define f_{t+1}, so we choose the closest match, i.e., a f_{t+1} maximizing its similarity:

$$f_{t+1} = arg\max_{f \in \mathcal{F}} f \cdot -\nabla_{f^T} L^{t+1}.$$

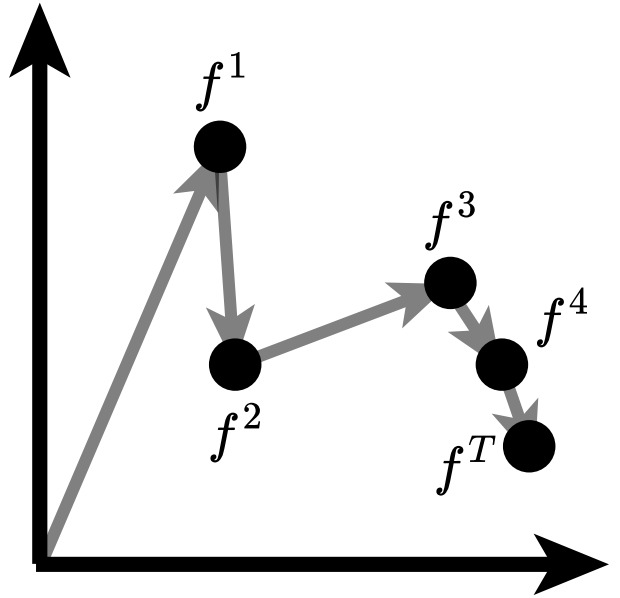


Boosting and iterative optimization: each model is added to the boosting model, thus moving across the model space.

The gradient in boosting: residuals

Università di Pisa

In boosting models, gradient of the loss gives us a search direction, but also a *residual*: practically, since each model is additive, gradients define directions as much as *residuals* we want to optimize. Thus, we fit models on datasets $(X, -\nabla L^t)$.



Boosting and iterative optimization: each model is added to the boosting model, thus moving across the model space.

Boosting: a most generic algorithm

UNIVERSITÀ DI PISA

Knowing how to learn a model, we can inductively define any boosting algorithm:

1. Initialize f_1
2. Find optimal direction $-\nabla_{f_t} L^t$
3. Find admissible direction f_t
4. Find learning step η_t
5. Learn f^t
6. Go to 2.

Boosting: a most generic algorithm

Università di Pisa

Knowing how to learn a model, we can inductively define any boosting algorithm:

1. Initialize f_1
2. Find optimal direction $-\nabla_{f_t} L^t$
3. Find admissible direction f_t
4. Find learning step η_t
5. Learn f^t
6. Go to 2.

Notes

- The function space could be anything: the space of Decision Trees (Gradient-Boosted Trees), of Logistic Regression (Logitboost), etc.
- Regularization is usually applied to f_t: allows to have weak learners, which helps with decreasing overfit

Boosting and its many flavors

- **Adaboost.** Uses an exponential loss, adapts η with a closed form search
- **Gradient-Boosted Trees.** Leverage Decision Trees as models
- **XGBoost.** Leverages trees, and uses a more robust loss approximation through the Hessian, rather than the gradient

Boosting and bagging: regularization

Regularization is performed directly on the weak learners (to make sure we keep variance high), but can be applied post-hoc too through pruning!

Boosting: computational complexity

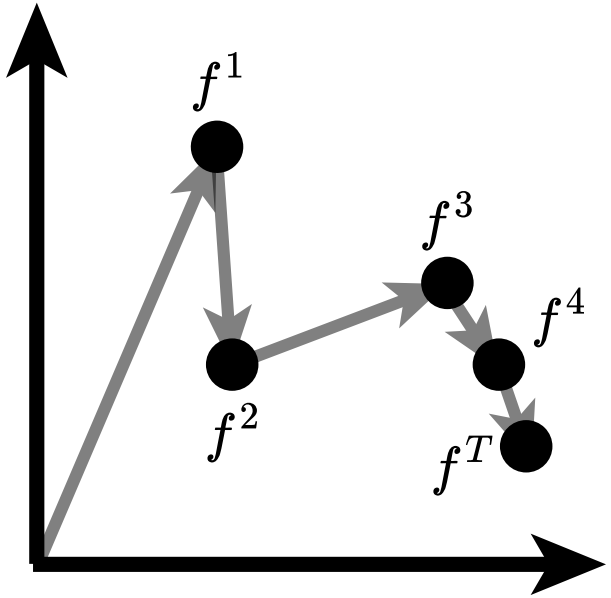
Unlike bagging, models are learned iteratively - see the general algorithm. Thus the computational complexity for k models is given by $\mathcal{O}(kc)$.

Remember: in asymptotical terms, constants are not considered.

Boosting

- Model-agnostic
- Given some relatively likely theoretical assumptions, has extremely low bias
- Unlikely to overfit
- Computationally quick

- Largely uninterpretable



Boosting: we iteratively refine the model f^T by exploring the function and loss space.

References

Università di Pisa

	Source
Bias, Variance	Deep Learning. I. Goodfellow, Y. Bengio, A. Courville. Sections 5.4
Boosting, Bagging	Deep Learning. I. Goodfellow, Y. Bengio, A. Courville. Sections 7.11
Random Forests	Random Forests

References to specific papers indicated in the slides footers'.